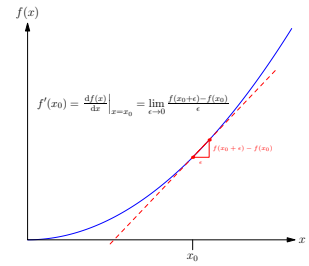


Differentiation, product and chain rules, vectors and matrices

Why Calculus?

- Calculus is a fundamental tool of mathematical analysis
- In machine learning differentiation is fundamental tool in optimisation
- Integration is an essential tool in taking expectations over continuous distributions
- Both differentiation and integration crop up elsewhere

1. Why Calculus?
2. Differentiation
3. Vector Calculus
4. Matrix Calculus

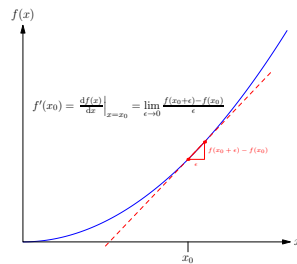


Back to Basics

- You have all done A-level maths so should be familiar with the rules of calculus
- But, it is easy to forget the rules and sometimes we use quite sophisticated tricks
- Although the sophisticated tricks really speed up calculations, it pays to be able to understand where these tricks come from

Outline

1. Why Calculus?
2. Differentiation
3. Vector Calculus
4. Matrix Calculus



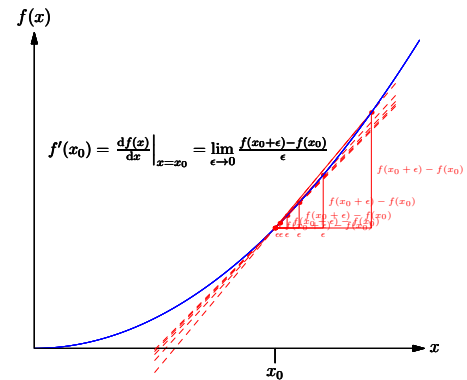
Polynomials

- $f(x) = x^2$

$$\frac{dx^2}{dx} = \lim_{\epsilon \rightarrow 0} \frac{(x+\epsilon)^2 - x^2}{\epsilon} = \lim_{\epsilon \rightarrow 0} \frac{(x^2 + 2\epsilon x + \epsilon^2) - x^2}{\epsilon} = \lim_{\epsilon \rightarrow 0} 2x + \epsilon = 2x$$
- $(x+\epsilon)^n = (x+\epsilon)(x+\epsilon)\cdots(x+\epsilon) = x^n + n\epsilon x^{n-1} + O(\epsilon^2)$

$$\frac{dx^n}{dx} = \lim_{\epsilon \rightarrow 0} \frac{(x+\epsilon)^n - x^n}{\epsilon} = \lim_{\epsilon \rightarrow 0} n x^{n-1} + O(\epsilon) = n x^{n-1}$$

Differentiation

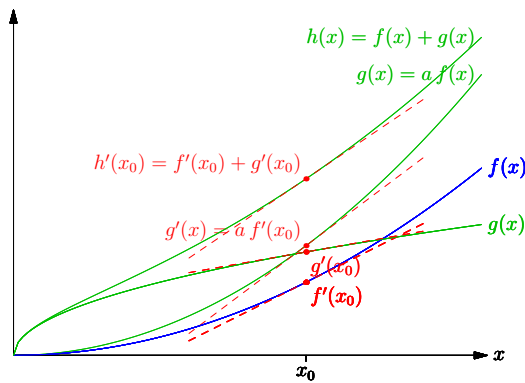


Linearity of derivatives

- Consider the derivative of $af(x) + bg(x)$

$$\begin{aligned} \frac{d(af(x) + bg(x))}{dx} &= \lim_{\epsilon \rightarrow 0} \frac{(af(x+\epsilon) + bg(x+\epsilon)) - (af(x) + bg(x))}{\epsilon} \\ &= \lim_{\epsilon \rightarrow 0} \frac{a(f(x+\epsilon) - f(x)) + b(g(x+\epsilon) - g(x))}{\epsilon} \\ &= \lim_{\epsilon \rightarrow 0} a \left(\frac{f(x+\epsilon) - f(x)}{\epsilon} \right) + b \left(\frac{g(x+\epsilon) - g(x)}{\epsilon} \right) \\ &= af'(x) + bg'(x) \end{aligned}$$
- Differentiation is a linear operation

Linearity in Pictures



Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

Chain Rule

- Recall $f(x + \epsilon) = f(x) + \epsilon f'(x) + O(\epsilon^2)$
- Let $h(x) = f(g(x))$
- Then

$$\begin{aligned} h(x + \epsilon) &= f(g(x + \epsilon)) = f(g(x) + \epsilon g'(x) + O(\epsilon^2)) \\ &= f(g(x)) + \epsilon g'(x) f'(g(x)) + O(\epsilon^2) \end{aligned}$$

- Thus

$$h'(x) = \lim_{\epsilon \rightarrow 0} \frac{h(x + \epsilon) - h(x)}{\epsilon} = g'(x) f'(g(x))$$

- This is the famous **chain rule**. Together with the product rule it means you can differentiate almost everything

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

Inverse functions

- Suppose $g(y) = f^{-1}(y)$ is the inverse of $f(x)$ in the sense that $g(f(x)) = f^{-1}(f(x)) = x$
- Using the chain rule

$$\frac{dg(f(x))}{dx} = f'(x) g'(f(x)) = 1$$

since $g(f(x)) = x$

- So $g'(f(x)) = 1/f'(x)$
- Writing $y = f(x)$ so that $x = f^{-1}(y) = g(y)$ we find $g'(y) = 1/f'(g(y))$ that is

$$\frac{dg(y)}{dy} = \frac{1}{f'(g(y))} \quad \frac{df^{-1}(y)}{dy} = \frac{1}{f'(f^{-1}(y))}$$

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

Functions of Exponentials

- What about $f(x) = e^{cx}$

$$\frac{de^{cx}}{dx} = \frac{de^{cx}}{dcx} \frac{dcx}{dx} = ce^{cx}$$

- More generally using the chain rule

$$\frac{de^{g(x)}}{dx} = g'(x) e^{g(x)}$$

- Also $a^{bc} = (a^b)^c$ (that is we multiply a together $b \times c$ times)

$$\frac{da^x}{dx} = \frac{d(e^{\ln(a)x})}{dx} = \frac{de^{\ln(a)x}}{dx} = \ln(a) e^{\ln(a)x} = \ln(a) a^x$$

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

Product Rule

- Recall $f(x + \epsilon) = f(x) + \epsilon f'(x) + O(\epsilon^2)$
- If $h(x) = f(x)g(x)$

$$\begin{aligned} h'(x) &= \lim_{\epsilon \rightarrow 0} \frac{f(x + \epsilon)g(x + \epsilon) - f(x)g(x)}{\epsilon} \\ &= \lim_{\epsilon \rightarrow 0} \frac{(f(x) + \epsilon f'(x) + O(\epsilon^2))(g(x) + \epsilon g'(x) + O(\epsilon^2)) - f(x)g(x)}{\epsilon} \\ &= \lim_{\epsilon \rightarrow 0} \frac{\epsilon(f'(x)g(x) + f(x)g'(x)) + O(\epsilon^2)}{\epsilon} = f'(x)g(x) + f(x)g'(x) \end{aligned}$$

- This is the **product rule**

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

More on chain rules

- We can also write the chain rule as

$$\frac{df(g(x))}{dx} = \frac{df(g)}{dg} \frac{dg(x)}{dx}$$

- Sometimes this is neater or easier to remember

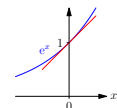
$$\begin{aligned} \frac{d e^{\cos(x^2)}}{dx} &= \frac{d e^{\cos(x^2)}}{d \cos(x^2)} \frac{d \cos(x^2)}{dx^2} \frac{dx^2}{dx} \\ &= e^{\cos(x^2)} (-\sin(x^2)) 2x \\ &= -2x \sin(x^2) e^{\cos(x^2)} \end{aligned}$$

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

Exponentials

- Note that $a^{b+c} = a^b a^c$ (that is we multiply a together $b + c$ times)

- Now $e^\epsilon \approx (1 + \epsilon)$



- But $e^{x+\epsilon} = e^x e^\epsilon = e^x (1 + \epsilon + O(\epsilon^2)) = e^x + \epsilon e^x + O(\epsilon^2)$

$$\frac{de^x}{dx} = \lim_{\epsilon \rightarrow 0} \frac{e^{x+\epsilon} - e^x}{\epsilon} = \lim_{\epsilon \rightarrow 0} \frac{\epsilon e^x + O(\epsilon^2)}{\epsilon} = e^x$$

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

Natural Logarithms

- The natural logarithm is defined as the inverse of e^x

$$\ln(e^x) = x \quad e^{\ln(y)} = y$$

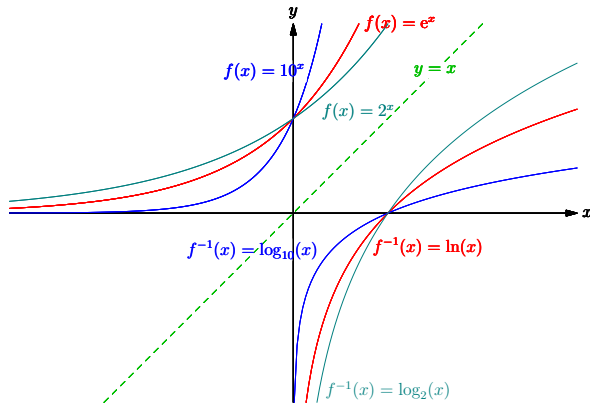
- Recall that if $g(y) = f^{-1}(y)$ then $g'(y) = 1/f'(g(y))$

- Consider $g(y) = \ln(y)$ and $f(x) = e^x$ (with $f'(x) = e^x$)

$$\frac{d \ln(y)}{dy} = \frac{1}{e^{\ln(y)}} = \frac{1}{y}$$

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

Exponentials and Logarithms



Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

Derivatives in High Dimensions

- When working with functions $f: \mathbb{R}^n \rightarrow \mathbb{R}$ in many dimensions then there will typically be different derivative in different directions
- To compute the derivative in a direction $\mathbf{u} \in \mathbb{R}^n$ (where $\|\mathbf{u}\| = 1$) at a point $\mathbf{x} \in \mathbb{R}^n$ we use

$$\partial_{\mathbf{u}} F(\mathbf{x}) = \lim_{\epsilon \rightarrow 0} \frac{f(\mathbf{x} + \epsilon \mathbf{u}) - f(\mathbf{x})}{\epsilon}$$

- If $\mathbf{u} = \delta_i = (0, \dots, 0, 1, 0, \dots, 0)^T$ (i.e. $u_i = 1$) then

$$\frac{\partial f(\mathbf{x})}{\partial x_i} = \lim_{\epsilon \rightarrow 0} \frac{f(\mathbf{x} + \epsilon \delta_i) - f(\mathbf{x})}{\epsilon}$$

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

Computing Gradients 1

- We can compute the gradient by writing out $f(\mathbf{x})$ componentwise and performing the partial derivative with respect to x_i

$$\begin{aligned} \nabla \mathbf{w}^T \mathbf{M} \mathbf{w} &= \begin{pmatrix} \frac{\partial}{\partial w_1} \\ \frac{\partial}{\partial w_2} \\ \frac{\partial}{\partial w_3} \\ \vdots \end{pmatrix} \sum_{i,j} w_i M_{ij} w_j = \begin{pmatrix} \sum_j M_{1j} w_j + \sum_i w_i M_{i1} \\ \sum_j M_{2j} w_j + \sum_i w_i M_{i2} \\ \sum_j M_{3j} w_j + \sum_i w_i M_{i3} \\ \vdots \end{pmatrix} \\ &= \mathbf{M} \mathbf{w} + \mathbf{M}^T \mathbf{w} = (\mathbf{M} + \mathbf{M}^T) \mathbf{w} \end{aligned}$$

- It is tedious to compute these things component-wise, but when you need to understand what is going on then go back to the basics

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

Function Composition

- The gradient tells us about the change in a function value when we perturb an input $\mathbf{x}$ by a small amount in the directions $\mathbf{u}$

$$f(\mathbf{x} + \epsilon \mathbf{u}) = f(\mathbf{x}) + \epsilon \mathbf{u}^T \nabla f(\mathbf{x}) + O(\epsilon^2)$$

- What happens when we compose functions: $h(\mathbf{x}) = f(g(\mathbf{x}))$? Note that,

$$\mathbf{x} \in \mathbb{R}^n \quad \mathbf{g}: \mathbb{R}^n \rightarrow \mathbb{R}^m \quad f: \mathbb{R}^m \rightarrow \mathbb{R} \quad h: \mathbb{R}^n \rightarrow \mathbb{R}$$

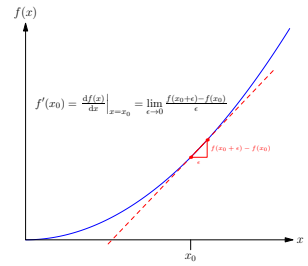
- The vector valued function $\mathbf{g}(\mathbf{x})$ can be seen as m independent

$$\text{function } \mathbf{g}(\mathbf{x}) = \begin{pmatrix} g_1(\mathbf{x}) \\ g_2(\mathbf{x}) \\ \vdots \\ g_m(\mathbf{x}) \end{pmatrix}$$

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

Outline

- Why Calculus?
- Differentiation
- Vector Calculus
- Matrix Calculus



Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

Taylor

- If we expand $f(\mathbf{x} + \epsilon \mathbf{u})$ to first order in ϵ

$$f(\mathbf{x} + \epsilon \mathbf{u}) = f(\mathbf{x}) + \epsilon \mathbf{u}^T \mathbf{g}(\mathbf{x}) + O(\epsilon^2)$$

$$\text{then } g_i(\mathbf{x}) = \frac{\partial f(\mathbf{x})}{\partial x_i}$$

- Recall we defined the vector of first order derivatives of $f(\mathbf{x})$ to be the gradient

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \frac{\partial f(\mathbf{x})}{\partial x_2} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_n} \end{pmatrix}$$

- Thus

$$f(\mathbf{x} + \epsilon \mathbf{u}) = f(\mathbf{x}) + \epsilon \mathbf{u}^T \nabla f(\mathbf{x}) + O(\epsilon^2)$$

This is the start of the high-dimensional Taylor expansion

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

Computing Gradients 2

- A slicker way is just to expand $f(\mathbf{x} + \epsilon \mathbf{u})$

- Consider $f(\mathbf{x}) = \mathbf{x}^T \mathbf{M} \mathbf{x} + \mathbf{a}^T \mathbf{x}$

$$\begin{aligned} f(\mathbf{x} + \epsilon \mathbf{u}) &= (\mathbf{x} + \epsilon \mathbf{u})^T \mathbf{M} (\mathbf{x} + \epsilon \mathbf{u}) + \mathbf{a}^T (\mathbf{x} + \epsilon \mathbf{u}) \\ &= f(\mathbf{x}) + \epsilon (\mathbf{u}^T \mathbf{M} \mathbf{x} + \mathbf{x}^T \mathbf{M} \mathbf{u} + \mathbf{a}^T \mathbf{u}) + O(\epsilon^2) \\ &= f(\mathbf{x}) + \epsilon \mathbf{u}^T (\mathbf{M} \mathbf{x} + \mathbf{M}^T \mathbf{x} + \mathbf{a}) + O(\epsilon^2) \end{aligned}$$

$$\text{using } \mathbf{x}^T \mathbf{M} \mathbf{u} = \mathbf{u}^T \mathbf{M}^T \mathbf{x} \text{ and } \mathbf{a}^T \mathbf{u} = \mathbf{u}^T \mathbf{a}$$

- But $f(\mathbf{x} + \epsilon \mathbf{u}) = f(\mathbf{x}) + \epsilon \mathbf{u}^T \nabla f(\mathbf{x}) + O(\epsilon^2)$ so

$$\nabla f(\mathbf{x}) = \mathbf{M} \mathbf{x} + \mathbf{M}^T \mathbf{x} + \mathbf{a}$$

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

Jacobians

- Perturbing $\mathbf{x}$ by $\epsilon \mathbf{u}$

$$g_i(\mathbf{x} + \epsilon \mathbf{u}) = g_i(\mathbf{x}) + \epsilon \mathbf{u}^T \nabla g_i(\mathbf{x}) + O(\epsilon^2)$$

so that

$$\mathbf{g}(\mathbf{x} + \epsilon \mathbf{u}) = \mathbf{g}(\mathbf{x}) + \epsilon \mathbf{J} \mathbf{u} + O(\epsilon^2)$$

where $\mathbf{J}$ is an $m \times n$ matrix with components $J_{ij} = \frac{\partial g_i(\mathbf{x})}{\partial x_j}$

- Note that $\mathbf{J} \mathbf{u}$ is a column vector of size m
- Ignoring terms of order ϵ^2

$$\begin{aligned} h(\mathbf{x} + \epsilon \mathbf{u}) &= f(\mathbf{g}(\mathbf{x} + \epsilon \mathbf{u})) = f(\mathbf{g}(\mathbf{x}) + \epsilon \mathbf{J} \mathbf{u}) \\ &= f(\mathbf{g}(\mathbf{x})) + \epsilon \mathbf{u}^T \mathbf{J}^T \nabla f(\mathbf{g}(\mathbf{x})) \end{aligned}$$

Adam Prügel-Bennett AICE3001/6002 Mathematics of Machine Learning <https://ecs-vlc.github.io/aice3001/>

- Using $h(\mathbf{x}) = f(\mathbf{g}(\mathbf{x}))$

$$h(\mathbf{x} + \epsilon \mathbf{u}) = h(\mathbf{x}) + \epsilon \mathbf{u}^\top \mathbf{J}^\top \nabla_{\mathbf{g}} f(\mathbf{g}(\mathbf{x}))$$

- We get that

$$\nabla_{\mathbf{x}} h(\mathbf{x}) = \nabla_{\mathbf{x}} f(\mathbf{g}(\mathbf{x})) = \mathbf{J}^\top \nabla_{\mathbf{g}} f(\mathbf{g}(\mathbf{x}))$$

- Componentwise

$$\frac{\partial h(\mathbf{x})}{\partial x_i} = \frac{\partial f(\mathbf{g}(\mathbf{x}))}{\partial x_i} = \sum_{j=1}^m \frac{\partial g_j(\mathbf{x})}{\partial x_i} \frac{\partial f(\mathbf{g}(\mathbf{x}))}{\partial g_j}$$

Differentiating Matrices

- Often we have loss functions with respect to a matrix $\mathbf{W}$, e.g.

$$L(\mathbf{W}) = (\mathbf{a}^\top \mathbf{W} \mathbf{b} - c)^2$$

- We might want to find the minimum with respect to $\mathbf{W}$
- This occurs at a point $\mathbf{W}^*$ where $L(\mathbf{W})$ does not increase as we change $\mathbf{W}$ in any way
- That is, we seek a $\mathbf{W}^*$ such that, for any matrices $\mathbf{U}$

$$L(\mathbf{W}^* + \epsilon \mathbf{U}) - L(\mathbf{W}^*) = O(\epsilon^2)$$

Example

- Suppose

$$L(\mathbf{W}) = (\mathbf{a}^\top \mathbf{W} \mathbf{b} - c)^2$$

then

$$\begin{aligned} L(\mathbf{W} + \epsilon \mathbf{U}) &= (\mathbf{a}^\top (\mathbf{W} + \epsilon \mathbf{U}) \mathbf{b} - c)^2 = (\mathbf{a}^\top \mathbf{W} \mathbf{b} + \epsilon \mathbf{a}^\top \mathbf{U} \mathbf{b} - c)^2 \\ &= L(\mathbf{W}) + 2\epsilon (\mathbf{a}^\top \mathbf{W} \mathbf{b} - c) (\mathbf{a}^\top \mathbf{U} \mathbf{b}) + O(\epsilon^2) \end{aligned}$$

- Now

$$\mathbf{a}^\top \mathbf{U} \mathbf{b} = \sum_{ij} a_i U_{ij} b_j = \sum_{ij} U_{ji} a_j b_i = \text{tr} \mathbf{U}^\top \mathbf{a} \mathbf{b}^\top$$

$$\text{Thus } \frac{\partial L(\mathbf{W})}{\partial \mathbf{W}} = 2(\mathbf{a}^\top \mathbf{W} \mathbf{b} - c) \mathbf{a} \mathbf{b}^\top$$

Quick Matrix Differentiation

- Let

$$\partial_{\mathbf{U}} f(\mathbf{X}) = \lim_{\epsilon \rightarrow 0} \frac{f(\mathbf{X} + \epsilon \mathbf{U}) - f(\mathbf{X})}{\epsilon} = \text{tr} \mathbf{U}^\top \frac{\partial f(\mathbf{X})}{\partial \mathbf{X}}$$

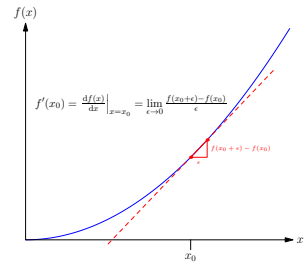
- E.g.

$$\begin{aligned} \partial_{\mathbf{U}} \text{tr} \mathbf{A} \mathbf{X} \mathbf{B} &= \lim_{\epsilon \rightarrow 0} \frac{1}{\epsilon} (\text{tr} \mathbf{A} (\mathbf{X} + \epsilon \mathbf{U}) \mathbf{B} - \text{tr} \mathbf{A} \mathbf{X} \mathbf{B}) \\ &= \text{tr} \mathbf{A} \mathbf{U} \mathbf{B} = \text{tr} \mathbf{B}^\top \mathbf{U}^\top \mathbf{A}^\top = \text{tr} \mathbf{U}^\top \mathbf{A}^\top \mathbf{B}^\top \end{aligned}$$

thus

$$\frac{\partial \text{tr} \mathbf{A} \mathbf{X} \mathbf{B}}{\partial \mathbf{X}} = \mathbf{A}^\top \mathbf{B}^\top$$

- Why Calculus?
- Differentiation
- Vector Calculus
- Matrix Calculus



Generalised Gradient

- We can generalise the idea of gradient to matrices

$$\frac{\partial L(\mathbf{W})}{\partial \mathbf{W}} = \begin{pmatrix} \frac{\partial L(\mathbf{W})}{\partial W_{11}} & \frac{\partial L(\mathbf{W})}{\partial W_{12}} & \dots & \frac{\partial L(\mathbf{W})}{\partial W_{1m}} \\ \frac{\partial L(\mathbf{W})}{\partial W_{21}} & \frac{\partial L(\mathbf{W})}{\partial W_{22}} & \dots & \frac{\partial L(\mathbf{W})}{\partial W_{2m}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial L(\mathbf{W})}{\partial W_{n1}} & \frac{\partial L(\mathbf{W})}{\partial W_{n2}} & \dots & \frac{\partial L(\mathbf{W})}{\partial W_{nm}} \end{pmatrix}$$

- From an identical argument we used for vectors

$$L(\mathbf{W} + \epsilon \mathbf{U}) = L(\mathbf{W}) + \epsilon \text{tr} \mathbf{U}^\top \frac{\partial L(\mathbf{W})}{\partial \mathbf{W}} + O(\epsilon^2)$$

where

$$\text{tr} \mathbf{U}^\top \mathbf{G} = \sum_i [\mathbf{U}^\top \mathbf{G}]_{ii} = \sum_{ij} U_{ji} G_{ji} = \sum_{ij} U_{ij} G_{ij} = \langle \mathbf{U}, \mathbf{G} \rangle$$

Traces

- The trace of a matrix is the sum of its diagonal elements

$$\text{tr} \mathbf{A} = \text{tr} \mathbf{A}^\top = \sum_i A_{ii}$$

- Clearly $\text{tr} \mathbf{c} \mathbf{A} = c \text{tr} \mathbf{A}$

- Also $\text{tr}(\mathbf{A} + \mathbf{B}) = \text{tr} \mathbf{A} + \text{tr} \mathbf{B}$

- We note that

$$\text{tr} \mathbf{A} \mathbf{B} = \sum_{i,j} A_{ij} B_{ji} = \sum_{i,j} B_{ij} A_{ji} = \text{tr} \mathbf{B} \mathbf{A}$$

- It follows that

$$\text{tr} \mathbf{A} \mathbf{B} \mathbf{C} \mathbf{D} = \text{tr} \mathbf{D} \mathbf{A} \mathbf{B} \mathbf{C} = \text{tr} \mathbf{C} \mathbf{D} \mathbf{A} \mathbf{B} = \text{tr} \mathbf{B} \mathbf{C} \mathbf{D} \mathbf{A}$$

Summary

- With care you can differentiate most expressions
- The chain and product rule are incredibly powerful tools
- We can generalise differentiation to vectors and matrices
- There are a number of surprisingly useful results see [The Matrix Cookbook](#)
- When we look at [integration](#) it gets harder